Appendix C - code (CSS files not included - can be found in product folder)

Server.js - defines all available routes and core functions, enabling communication between the front-end interface and the back-end logic while coordinating operations within the web application.

```
require('dotenv').config();
const express = require('express');
const bodyParser = require('body-parser');
const mysql = require('mysql');
const bcrypt = require('bcrypt');
const session = require('express-session');
const nodemailer = require('nodemailer');
const crypto = require('crypto');
const path = require('path');
const { jsPDF } = require('jspdf');
const { createCanvas } = require('canvas');



const app = express();
const saltRounds = 10;

// Middleware
app.use(bodyParser.urlencoded({ extended: true, limit: '50mb' }));
app.use(bodyParser.json({ limit: '50mb' }));

app.use(express.static('public'));

// app.use(session({
//     secret: 'your-secret-key',
//     //A cryptographic string used to sign the session ID cookie
//      // ensures integrity so clients cannot tamper with their own session ID.

//     resave: false,
//     // Prevents session data from being re-saved to the store if nothing has
changed →
//     // improves performance and reduces unnecessary writes.

//     saveUninitialized: true
//     //Forces a new but empty session to be stored even if no data is set →
//     // ensures that a unique session ID is issued for every visitor, which is later
be populated on login.
```

```
// }));

// 🔐 Persistent session management using MySQL store
const MySQLStore = require('express-mysql-session')(session);

const sessionStore = new MySQLStore({
 host: 'localhost',
 user: 'root',
 password: process.env.DB_PASSWORD,
 database: 'LoginDB',
 clearExpired: true,
 checkExpirationInterval: 900000, // clear expired every 15 mins
 expiration: 24 * 60 * 60 * 1000  // 1 day session lifespan
});

app.use(session({
 key: 'user_sid',
 secret: 'your-secret-key', // use a long, random key in .env ideally
 store: sessionStore,
 resave: false,
 saveUninitialized: false, // prevents saving empty sessions
 cookie: {
   maxAge: 24 * 60 * 60 * 1000, // 1 day
   httpOnly: true, // protects from JS access
   secure: false   // set true only if using HTTPS
 }
}));

app.use((req, res, next) => {
   console.log('Session data:', req.session);
   next();
});

// MySQL Connection
const db = mysql.createConnection({
   host: 'localhost',
   user: 'root',
   password: process.env.DB_PASSWORD,
   database: 'LoginDB' // The database password is stored in a local file called .env
as it shouldn't be shared.
});
```

```
//The backend communicates with the relational database using the MySQL dependency,
which establishes access to the library.


db.connect((err) => {
   if (err) {
       console.error('Database connection failed:', err);
       return;
   }
   console.log('Connected to MySQL database');
});

// Nodemailer
const transporter = nodemailer.createTransport({
   service: 'gmail',
   auth: {
       user: 'blissfulbarfi@gmail.com',
       pass: 'txsl fpmv qtjy ziym'
   }
});

// Login + Role Redirection
app.use(express.json());
app.post('/login', (req, res) => {
 const { username, password } = req.body;

 const query = 'SELECT * FROM users WHERE username = ?';
 db.query(query, [username], async (err, results) => {
   if (err) return res.status(500).send('Database error.');
   if (results.length === 0) return res.send('Error: User not found.');

   const user = results[0];

   // ✅ Compare entered password with hashed password from DB
   try {
     const isMatch = await bcrypt.compare(password, user.password);
     if (!isMatch) return res.send('Error: Incorrect password.');
   } catch (bcryptErr) {
     console.error('Password comparison error:', bcryptErr);
     return res.status(500).send('Internal error during authentication.');
   }
```

```
    // ✅ Store essential user details in the session
    req.session.user = {
      id: user.id,            // unique user identifier
      username: user.username, // username displayed in dashboard
      role: user.role         // used for role-based access control
    };

    // ✅ Explicitly save the session to ensure persistence before redirect
    req.session.save((err) => {
      if (err) {
        console.error('Session save error:', err);
        return res.status(500).send('Error saving session.');
      }

      console.log('✅ Session saved for:', req.session.user.username);

      // ✅ Redirect user to their respective dashboard
      if (user.role === 'admin') return res.redirect('/admin');
      else if (user.role === 'General Employee') return
res.redirect('/employee-dashboard');
      else if (user.role === 'Department Head') return
res.redirect('/department-head');
      else return res.send('Login successful, but no dashboard defined for your
role.');
    });
  });
});
function ensureAuthenticated(req, res, next) {
  if (req.session && req.session.user) {
    return next();
  } else {
    return res.status(401).send('Session expired. Please log in again.');
  }
}

// Admin Routes
function ensureAuthenticated(req, res, next) {
  if (req.session.user) return next();
  res.send('You must log in to access this page.');
}
```

```
app.get('/admin', (req, res) => {  //Checks if a user is logged in and has the role
'admin'
/*
When a request is made to /admin, the server checks if the session exists and if the
role is equal to admin.
Only then is the requested file served; otherwise, access is denied.
*/
   if (req.session.user && req.session.user.role === 'admin') {
       //check if the user is stored as an admin in the database
       res.sendFile(path.join(__dirname, 'public', 'admin.html'));
       //when the if condition passes, the admin dashboard webpage is sent to the
user's browser

   } else {
       res.send('Access denied. Admins only.');
       // If condition fails, unauthorised access is denied immediately
   }
   // This route protects sensitive admin dashboard from being accessed by non-admin
users


});

// app.get('/admin-dashboard', (req, res) => {
//     res.sendFile(path.join(__dirname, 'public', 'admin-dashboard.html'));
// });

// //  Admin dashboard data route
// app.get('/admin-dashboard-data', (req, res) => {
//   if (req.session.user && req.session.user.role === 'admin') {
//     const departmentsQuery = `
//       SELECT d.department_id, d.department_name, d.requested_budget,
d.admin_comments,
//              d.budget_status, u.username AS department_head
//       FROM departments d
//       LEFT JOIN users u ON d.department_head_id = u.id
//     `;

//     const expendituresQuery = `
//       SELECT department_id, SUM(expenditure_amount) AS actual_spent
//       FROM expenditures
//       GROUP BY department_id
```

```
//     `;

//     db.query(departmentsQuery, (err, departmentsResults) => {
//       if (err) return res.status(500).send('Error fetching departments data.');

//       db.query(expendituresQuery, (err, expendituresResults) => {
//         if (err) return res.status(500).send('Error fetching expenditures data.');

//         res.json({ //the json object contains both departmental data and aggregated
expenditures
//           departments: departmentsResults,
//           expenditures: expendituresResults
//         });
//         /*
//         For each of the expenditure records that are retrieved, the details
(department_ID, department_name, etc.)
//          are requested from the database.
//       */});
//     });
//   } else {
//     res.send('Access denied. Admins only.');
//   }
// });

// Route for Admin Dashboard Page
app.get('/admin-dashboard', (req, res) => {
 if (req.session.user && req.session.user.role === 'admin') {
   // Serve the actual dashboard HTML file
   res.sendFile(path.join(__dirname, 'public', 'admin-dashboard.html'));
 } else {
   res.status(403).send('Access denied. Admins only.');
 }
});


app.get('/admin-dashboard-data', (req, res) => {
 /*
 grouped aggregation -  all expenditures are grouped by their department_id instead of
                        sending separate requests for each department's expenditures.

 benefits -  This design reduces the total number of HTTP calls the frontend needs to
make, improving performance and responsiveness
```

```
            at runtime.
 drawbacks - it also means that the initial payload is larger, which may increase the
initial page load time — sacrificing latency
            for bandwidth.

 */
 if (req.session.user && req.session.user.role === 'admin') {
   const query = `
     SELECT d.department_id, d.department_name, d.requested_budget, d.admin_comments,
            d.budget_status, u.username AS department_head,
            COALESCE(SUM(e.expenditure_amount), 0) AS actual_spent
     FROM departments d
     LEFT JOIN users u ON d.department_head_id = u.id
     LEFT JOIN expenditures e ON d.department_id = e.department_id
     GROUP BY d.department_id, d.department_name, d.requested_budget,
              d.admin_comments, d.budget_status, u.username
   `;
   /*department_id is the foreign key in the expenditures table, and is used to sort
expenditures by department.
   user_id is the foreign id in the department table, and is used to show which user
is the department head for the
   expenditure-department. hence all relevant data can be displayed.
   */
   db.query(query, (err, results) => {
     if (err) return res.status(500).send('Error fetching dashboard data.');
     res.json(results || []);
   });
 } else {
   res.send('Access denied. Admins only.');
 }
});

// === Update Comment === //
app.post('/update-comment', (req, res) => {
 console.log('/update-comment body:', req.body);
 const { department_id, comment } = req.body;
 if (!department_id || typeof comment !== 'string') {
   return res.status(400).send('Invalid payload');
 }

 const query = 'UPDATE departments SET admin_comments = ? WHERE department_id = ?';
 db.query(query, [comment, department_id], (err, result) => {
```

```
    if (err) {
      console.error('DB error on update-comment:', err);
      return res.status(500).send('Error updating comment.');
    }
    if (result.affectedRows === 0) {
      return res.status(404).send('Department not found');
    }

    // Notify department head (still resolve even if email fails)
    notifyDeptHead(department_id)
      .then(() => res.sendStatus(200))
      .catch(e => {
        console.error("Notification error (comment):", e);
        res.sendStatus(200);
      });
  });
});

// Update status route (fixed braces + validation)
app.post('/update-status', (req, res) => {
  console.log('/update-status body:', req.body);
  const { department_id, status } = req.body;

  // validate status is A/D/P
  if (!department_id || !['A','D','P'].includes(status)) {
    return res.status(400).send('Invalid payload or status (must be A, D, or P)');
  }

  const query = 'UPDATE departments SET budget_status = ? WHERE department_id = ?';
  db.query(query, [status, department_id], (err, result) => {
    if (err) {
      console.error('DB error on update-status:', err);
      return res.status(500).send('Error updating status.');
    }
    if (result.affectedRows === 0) {
      return res.status(404).send('Department not found');
    }

    // Notify department head (still succeed even if notification fails)
    notifyDeptHead(department_id)
      .then(() => res.sendStatus(200))
      .catch(e => {
```

```
      console.error("Notification error (status):", e);
      res.sendStatus(200);
    });
 });
});


function notifyDeptHead(department_id) {
 return new Promise((resolve, reject) => {
   const query = `
     SELECT d.department_name, d.requested_budget, d.budget_status, d.admin_comments,
            u.email, u.username AS dept_head_name
     FROM departments d
     JOIN users u ON d.department_head_id = u.id
     WHERE d.department_id = ?
     LIMIT 1
   `;

   db.query(query, [department_id], (err, results) => {
     if (err) return reject(err);
     if (!results || results.length === 0) return reject(new Error('Department not
found'));

     const row = results[0];

     // ✅ Decide email message type
     let messageLine = "Your recent budget request has been reviewed and has been
changed.";

     if (row.budget_status) {
       const status = row.budget_status.toLowerCase();
       if (status === "approved") {
         messageLine = "Your recent budget request has been reviewed and has been
approved.";
       } else if (status === "denied") {
         messageLine = "Your recent budget request has been reviewed and has not been
approved.";
       }
     }

     // ✅ Build email
     const mailOptions = {
```

```
      from: 'YOUR_GMAIL@gmail.com',
      to: row.email,
      subject: 'Budget Request Update',
      text: `Hello ${row.dept_head_name},

${messageLine}

Department: ${row.department_name}
Requested Amount: ${row.requested_budget}
Status: ${row.budget_status || '—'}
Comments: ${row.admin_comments || '—'}

Regards,
Your Budget Tracking System`
    };

    transporter.sendMail(mailOptions, (mailErr, info) => {
      if (mailErr) return reject(mailErr);
      console.log("Email sent:", info.response);
      resolve();
    });
  });
 });
}

// === Update Status === //
app.post('/update-status', (req, res) => {
 const { department_id, status } = req.body;
 const query = 'UPDATE departments SET budget_status = ? WHERE department_id = ?';
 db.query(query, [status, department_id], (err) => {
  if (err) return res.status(500).send('Error updating status.');

  // Notify department head
  notifyDeptHead(department_id)
    .then(() => res.sendStatus(200))
    .catch(e => {
      console.error("Notification error:", e);
      res.sendStatus(200); // still succeed for admin
    });
 });
});
```

```
// //testing testing delete after

// app.get('/test-mail', (req, res) => {
//     transporter.sendMail({
//         from: 'blissfulbarfi@gmail.com',
//         to: 'sneha.u.gautam@gmail.com',
//         subject: 'Test Email',
//         text: 'If you got this, email is working.',
//     }, (error, info) => {
//         if (error) {
//             console.error('Mail error:', error);
//             return res.send('Failed to send test email');
//         }
//         res.send('Test email sent successfully');
//     });
// });

// User Management Routes
app.get('/user-management', (req, res) => {
   res.sendFile(path.join(__dirname, 'public', 'user-management.html'));
});

app.get('/api/users', (req, res) => {
 if (req.session.user && req.session.user.role === 'admin') {
   const query = `
     SELECT u.id, u.username, u.role, d.department_name
     FROM users u
     LEFT JOIN departments d ON u.department_id = d.department_id
   `;
   db.query(query, (err, results) => {
     if (err) return res.status(500).send('Error fetching users.');
     res.json(results);
   });
 } else {
   res.status(403).send('Access denied.');
 }
});


app.post('/delete-user', (req, res) => {//this route handles administrator-initiated
requests to remove a user from the users table.
```

```
    const { id } = req.body;
    const query = 'DELETE FROM users WHERE id = ?';
    /* this is hard deletion - the user is removed permanently, meaning historical data
associated with that user (e.g., budget submissions)
                              may lose referential context.
     an alternate solution is soft deletion - (e.g., is_active = false), which
preserves referential integrity at the cost of
                                                 increased storage
     for this system -  hard deletion was appropriate, as administrators explicitly
requested the ability to purge inactive
                         accounts from the system.

    */
    db.query(query, [id], (err) => {
        if (err) return res.status(500).send('Error deleting user.');
        res.sendStatus(200);
    });
});

app.post('/edit-user', (req, res) => {
    const { id, name, department, role } = req.body;
    const query = 'UPDATE users SET name = ?, department = ?, role = ? WHERE id = ?';
    db.query(query, [name, department, role, id], (err, results) => {
        if (err) return res.status(500).send('Error updating user.');
        res.sendStatus(200);
    });
});

app.post('/api/users/add', async (req, res) => {
 const { username, email, password, role, department_name } = req.body;

 if (!username || !email || !password || !role || !department_name) {
    return res.status(400).send('Missing required fields.');
 }

 try {
    const hashedPassword = await bcrypt.hash(password, 10);

    // Step 1: Fetch department_id from department_name
    const getDeptQuery = 'SELECT department_id FROM departments WHERE department_name =
? LIMIT 1';
    db.query(getDeptQuery, [department_name], (err, deptResults) => {
```

```
    if (err) return res.status(500).send('Database error while fetching
department.');
    if (deptResults.length === 0) return res.status(400).send('Invalid department.');

    const department_id = deptResults[0].department_id;

    // Step 2: Insert new user
    const insertQuery = `
      INSERT INTO users (username, email, password, role, department_id)
      VALUES (?, ?, ?, ?, ?)
    `;
    db.query(insertQuery, [username, email, hashedPassword, role, department_id],
(insertErr) => {
      if (insertErr) {
        console.error('Error inserting user:', insertErr);
        return res.status(500).send('Error adding user.');
      }

      // Step 3: Send onboarding email
      sendOnboardingEmail(username, email, role, department_name)
        .then(() => {
          console.log(`✅ Onboarding guide sent to ${email}`);
          res.sendStatus(200);
        })
        .catch((e) => {
          console.error('Error sending onboarding email:', e);
          res.sendStatus(200); // Still succeed for the admin
        });
    });
  });
 } catch (error) {
  console.error('Error hashing password:', error);
  res.status(500).send('Server error.');
 }
});



// Other Routes
app.get('/budget-requests', (req, res) => {
  res.sendFile(path.join(__dirname, 'public', 'budget-requests.html'));
});
```

```
app.get('/budget-reports', (req, res) => {
   res.sendFile(path.join(__dirname, 'public', 'budget-reports.html'));
});

app.get('/employee-dashboard', (req, res) => {
   res.sendFile(path.join(__dirname, 'public', 'employee-dashboard.html'));
});

app.get('/manager-dashboard', (req, res) => {
   res.sendFile(path.join(__dirname, 'public', 'manager-dashboard.html'));
});

// ✅ New 3-Step Password Reset (Email Verification Code)

// Step 1: Request verification code
app.post('/request-reset', (req, res) => {
   const { username } = req.body;
   db.query('SELECT email FROM users WHERE username = ?', [username], (err, results)
=> {
       if (err || results.length === 0) return res.send('User not found');

       const email = results[0].email;
       const code = Math.floor(10000 + Math.random() * 90000); // 5-digit code

       req.session.resetUsername = username;
       req.session.verificationCode = code;

       const mailOptions = {
           from: 'blissfulbarfi@gmail.com',
           to: email,
           subject: 'Your Password Reset Code',
           text: `Your password reset verification code is: ${code}`
       };

       transporter.sendMail(mailOptions, (error) => {
           if (error) return res.send('Error sending email');
           res.redirect('/verify-code.html');
       });
   });
});
```

```
// Step 2: Verify the code
app.post('/verify-code', (req, res) => {
   const { code } = req.body;
   if (parseInt(code) === req.session.verificationCode) {
       req.session.codeVerified = true;
       res.redirect('/reset-password.html');
   } else {
       res.send('Invalid verification code.');
   }
});

// Step 3: Reset password
app.post('/reset-password', async (req, res) => {
   const { newPassword, confirmPassword } = req.body;

   if (!req.session.codeVerified || !req.session.resetUsername) {
       return res.send('Session expired or invalid access.');
   }

   if (newPassword !== confirmPassword) {
       return res.send('Passwords do not match.');
   }

   //code for updating the changed password if a user resets their password

   const hashedPassword = await bcrypt.hash(newPassword, saltRounds);
   //when registering a new administrator, the password entered by the user is first
passed through this function

   /*  'saltRounds' defines the cost factor (number of iterations) → more secure but
slower
                  - The saltRounds parameter controls the computational complexity of
the hashing process by introducing unique salts
                    and performing multiple iterations of hashing.
                  - This makes brute-force or rainbow-table attacks computationally
expensive. */
   db.query(
       'UPDATE users SET password = ? WHERE username = ?',
       // SQL UPDATE query to replace the old password with the new hashed password
       //The resulting hash is inserted into the users table, replacing the plaintext
value
```

```
        [hashedPassword, req.session.resetUsername], // Use parameterized values to
prevent SQL injection
        (err) => {
            if (err) return res.send('Error updating password.');
            req.session.destroy();
            // Destroy the session after password reset → prevents reuse of old session
            res.send('Password successfully updated.');
             // Send confirmation response to the client
        }
    );
});

//API for Budget Reports (THIS IS WHAT U ADD BACK IF THE CURRENT CODE GOES WRONG)
app.get('/api/budget-reports-data', (req, res) => {
 const query = `
    SELECT d.department_id, d.department_name, d.requested_budget, d.admin_comments,
d.budget_status,
           u.username AS department_head
    FROM departments d
    LEFT JOIN users u ON d.department_head_id = u.id
 `;
 db.query(query, (err, results) => {
    if (err) return res.status(500).send('Database error fetching reports');
    res.json(results);
 });
});




// API for User Management
app.get('/api/users', (req, res) => {
    const sql = `
        SELECT u.id AS id, u.username, u.role, d.department_name
        FROM users u
        LEFT JOIN departments d ON u.department_id = d.department_id
    `;
    db.query(sql, (err, results) => {
        if (err) {
            console.error('SQL Error:', err);
```

```
            return res.status(500).json({ error: 'Database error' });
        }
        res.json(results);
    });
});

// Update user details
app.post('/api/users/update', (req, res) => {
    const { id, username, role, department_name } = req.body;
    const getDeptIdSql = 'SELECT department_id FROM departments WHERE department_name =
?';

    db.query(getDeptIdSql, [department_name], (err, deptResult) => {
        if (err || deptResult.length === 0) return res.status(500).json({ error:
'Invalid department name' });

        const department_id = deptResult[0].department_id;
        const updateSql = 'UPDATE users SET username = ?, role = ?, department_id = ?
WHERE id = ?';

        db.query(updateSql, [username, role, department_id, id], (err2) => {
            if (err2) return res.status(500).json({ error: 'Failed to update user' });
            res.json({ success: true });
        });
    });
});

// Delete user
app.post('/api/users/delete', (req, res) => {
    const { id } = req.body;
    const deleteSql = 'DELETE FROM users WHERE id = ?';
    db.query(deleteSql, [id], (err) => {
        if (err) return res.status(500).json({ error: 'Failed to delete user' });
        res.json({ success: true });
    });
});




app.post('/email-budget-report', async (req, res) => {
```

```
try {
  const { pdfBase64, department_name } = req.body;

  // Get logged-in user's email from session
  const username = req.session.user?.username;
  if (!username) {
    console.log('No user in session');
    return res.status(401).send('Not logged in.');
  }

  db.query('SELECT email FROM users WHERE username = ?', [username], (err, results)
=> {
    if (err) {
      console.error('Database error:', err);
      return res.status(500).send('Database error.');
    }

    if (results.length === 0) {
      console.log('No user found');
      return res.status(404).send('User not found.');
    }

    const email = results[0].email;

    // Send email with attached PDF
    transporter.sendMail({
      from: 'blissfulbarfi@gmail.com',
      to: email,
      subject: `Budget Report for ${department_name}`,
      text: 'Attached is your requested budget report.',
      attachments: [{
        filename: `${department_name}_report.pdf`,
        content: Buffer.from(pdfBase64, 'base64'),
        contentType: 'application/pdf'
      }]
    }, (emailErr, info) => {
      if (emailErr) {
        console.error('Error sending email:', emailErr);
        return res.status(500).send('Error sending email.');
      }

      console.log('Email sent:', info.response);
```

```
      res.send('Email sent successfully!');
    });
  });
 } catch (err) {
  console.error('Unexpected error in /email-budget-report:', err);
  res.status(500).send('Internal server error.');
 }
});

function sendOnboardingEmail(username, email, role, department_name) {
 return new Promise((resolve, reject) => {
  // --- Role descriptions ---
  const roleDescriptions = {
    Admin: `As an Admin, you are responsible for overseeing all departmental budgets, approving requests, and ensuring organizational financial efficiency. You can manage users, track departmental expenditures, and generate comprehensive reports.`,
    'Department Head': `As a Department Head, your role is to plan, monitor, and justify your department's budget. You can edit spending plans, submit expenditure requests, and communicate with the Admin for approvals.`,
    'General Employee': `As a General Employee, you can view your department's financial information and track the status of expenditure requests. You play a key role in ensuring transparency and responsible spending within your department.`
  };

  // --- Department descriptions ---
  const departmentDescriptions = {
    HR: `The HR Department focuses on recruitment, employee engagement, and maintaining workplace culture. You'll oversee or contribute to initiatives related to hiring, training, and staff well-being.`,
    Finance: `The Finance Department is responsible for budget planning, audits, and ensuring optimal resource allocation. You'll monitor financial transactions and ensure compliance with internal policies.`,
    Marketing: `The Marketing Department manages campaigns, public relations, and brand strategy. You'll help the organization reach its goals by driving visibility and engagement.`,
    IT: `The IT Department ensures technological stability and data security across the organization. You'll maintain systems, troubleshoot issues, and implement digital solutions for efficiency.`,
    'Test Department': `The Test Department is a sandbox environment used for testing new workflows and system updates before they go live. You'll help evaluate functionality and report potential improvements.`
  };
```

```
   const roleText = roleDescriptions[role] || 'Your role involves active participation
in the organization's workflow.';
   const deptText = departmentDescriptions[department_name] || 'Your department plays
a vital role in the organization's operations.';

   // --- Construct email ---
   const mailOptions = {
     from: 'YOUR_GMAIL@gmail.com',
     to: email,
     subject: 'Welcome to the Budget Management System!',
     text: `Hello ${username},

Welcome to the Budget Management System! We're excited to have you on board.

Here's an overview of your role and department to help you get started:

**Your Role: ${role}**
${roleText}

**Your Department: ${department_name}**
${deptText}

To get started, log in using the credentials provided by your Admin.
Please remember to update your password once you first log in.

If you have any questions, reach out to your Department Head or Admin.

Best regards,
The Budget Management Team
`
   };

   // --- Send email ---
   transporter.sendMail(mailOptions, (err, info) => {
     if (err) return reject(err);
     console.log('Onboarding email sent:', info.response);
     resolve();
   });
 });
}
```

```
app.post('/register-admin', async (req, res) => {
 const { username, password, email } = req.body;
 const hashedPassword = await bcrypt.hash(password, saltRounds);
 const sql = 'INSERT INTO users (username, password, role, email) VALUES (?, ?, ?,
?)';
  db.query(sql, [username, hashedPassword, 'admin', email], (err) => {
   if (err) return res.status(500).send('Error registering admin.');
   res.send('Admin registered successfully');
 });
});


//Department Head Routes

app.get('/department-head', (req, res) => {
   if (req.session.user && req.session.user.role === 'Department Head') {
       res.sendFile(path.join(__dirname, 'public', 'department-head.html'));
   } else {
       res.send('Access denied. Department Heads only.');
   }
});

//department head dashboard route

app.get('/department-head-dashboard', (req, res) => {
 if (!req.session.user || req.session.user.role !== 'Department Head') {
   return res.status(403).send('Access denied. Department Heads only.');
 }
 res.sendFile(path.join(__dirname, 'public', 'department-head-dashboard.html'));
});

// Helper to resolve the Department Head's department_id
function getDeptHeadDepartmentId(req, callback) {
 // If you already store department_id in session, prefer that:
 if (req.session.user && req.session.user.department_id) {
   return callback(null, req.session.user.department_id);
 }
 // Fallback: look it up from users table using logged-in user's id
 const q = 'SELECT department_id FROM users WHERE id = ? LIMIT 1';
```

```
 db.query(q, [req.session.user.id], (err, rows) => {
   if (err) return callback(err);
   if (!rows || rows.length === 0) return callback(new Error('User not found'));
   callback(null, rows[0].department_id);
 });
}

// ==========================================
// API: Budget Approval Status (one row only)
// ==========================================
// app.get('/api/department-head/budget-status', (req, res) => {
//   if (!req.session.user || req.session.user.role !== 'Department Head') {
//     return res.status(403).send('Access denied. Department Heads only.');
//   }

//   getDeptHeadDepartmentId(req, (err, departmentId) => {
//     if (err) return res.status(500).send('Error resolving department.');

//     const query = `
//       SELECT department_name, department_head, requested_budget, budget_status,
admin_comments
//       FROM departments
//       WHERE department_id = ?
//       LIMIT 1
//     `;

//     db.query(query, [departmentId], (qErr, results) => {
//       if (qErr) return res.status(500).send('Error fetching budget status.');
//       // Return as an array (client expects array and forEach's it)
//       res.json(results || []);
//     });
//   });
// });

// ======================================
// API: Employees in this Department only
// ======================================
app.get('/api/department-head/employees', (req, res) => {
 if (!req.session.user || req.session.user.role !== 'Department Head') {//first checks
the user's role
   return res.status(403).send('Access denied. Department Heads only.');
```

```
   // If the logged-in user is not a Department Head, the server returns an HTTP 403
error, denying access to the resource.

 }
 getDeptHeadDepartmentId(req, (err, departmentId) => {  //Retrieve the department_id
for this head
   if (err) return res.status(500).send('Error resolving department.');
 /*
 Once resolved, the department_id is then passed into a parameterized SQL query
fetching only the employees
  belonging to the same department as the logged-in Department Head
 */
   const query = `
     SELECT u.id, u.username, u.role, u.email, d.department_name
     FROM users u
     JOIN departments d ON u.department_id = d.department_id
     WHERE u.department_id = ?
     ORDER BY u.username ASC
   `;// ✅ Query employees only within this department
   /*Once obtained from the database, the results are returned as a JSON object to the
frontend, which then renders them
   in the Department Head's dashboard.*/

   // Ensures Department Heads cannot query employees outside their department
   // Department ID is derived from the session, not from user input → prevents
tampering
   // Department ID is derived from the session, not from user input → prevents
tampering

   db.query(query, [departmentId], (qErr, results) => {
     if (qErr) return res.status(500).send('Error fetching employees.');
     res.json(results || []);
   });
 });
});

// ====================================================
// API: Update a user (only within this department)
// ====================================================
app.post('/api/department-head/employees/update', (req, res) => {
 if (!req.session.user || req.session.user.role !== 'Department Head') {
   return res.status(403).send('Access denied. Department Heads only.');
```

```
 }

 const { id, username, role, email } = req.body;
 if (!id || !username || !role || !email) {
   return res.status(400).send('Missing required fields.');
 }

 getDeptHeadDepartmentId(req, (err, departmentId) => {
   if (err) return res.status(500).send('Error resolving department.');

   const query = `
     UPDATE users
     SET username = ?, role = ?, email = ?
     WHERE id = ? AND department_id = ?
   `;
   db.query(query, [username, role, email, id, departmentId], (qErr, result) => {
     if (qErr) {
       console.error("❌ Employee update DB error:", qErr);
       return res.status(500).send('Error updating user.');
     }
     console.log("📌 Employee update result:", result);

     if (result.affectedRows === 0) {
       return res.status(403).send('Update denied or user not in your department.');
     }
     res.sendStatus(200);
   });
 });
});



// ====================================================
// API: Delete a user (only within this department)
// ====================================================
app.post('/api/department-head/employees/delete', (req, res) => {
 if (!req.session.user || req.session.user.role !== 'Department Head') {
   return res.status(403).send('Access denied. Department Heads only.');
 }

 const { id } = req.body;
 if (!id) return res.status(400).send('Missing user id.');
```

```
 getDeptHeadDepartmentId(req, (err, departmentId) => {
   if (err) return res.status(500).send('Error resolving department.');

   const query = `
     DELETE FROM users
     WHERE id = ? AND department_id = ?
   `;
   db.query(query, [id, departmentId], (qErr, result) => {
     if (qErr) return res.status(500).send('Error deleting user.');
     if (result.affectedRows === 0) {
       return res.status(403).send('Delete denied or user not in your department.');
     }
     res.sendStatus(200);
   });
 });
});

// ========================================
// API: Update budget (Department Head)
// ========================================
// ========================================
// API: Update budget (Department Head)
// ========================================
app.post('/api/department-head/budget/update', (req, res) => {
 console.log("📌 Budget update request received:", req.body, req.session.user);

 if (!req.session.user || req.session.user.role !== 'Department Head') {
   return res.status(403).send('Access denied. Department Heads only.');
 }

 const { requested_budget } = req.body;
 if (!requested_budget || isNaN(requested_budget)) {
   return res.status(400).send('Invalid budget amount.');
 }

 getDeptHeadDepartmentId(req, (err, departmentId) => {
   if (err) return res.status(500).send('Error resolving department.');

   const query = `UPDATE departments SET requested_budget = ? WHERE department_id =
?`;
   db.query(query, [requested_budget, departmentId], (qErr, result) => {
     if (qErr) {
```

```
      console.error("❌ Budget update DB error:", qErr);
      return res.status(500).send('Error updating budget.');
    }
    console.log("📌 MySQL update result:", result);  // ✅ correct place

    if (result.affectedRows === 0) {
      return res.status(404).send('Department not found or not allowed.');
    }

    res.sendStatus(200);
  });
 });
});



// ==============================
// Department Expenditures page
// ==============================
app.get('/department-head-expenditures', (req, res) => {
 if (!req.session.user || req.session.user.role !== 'Department Head') {
   return res.status(403).send('Access denied. Department Heads only.');
 }
 res.sendFile(path.join(__dirname, 'public', 'department-head-expenditures.html'));
});

// ==============================
// API: Get expenditures (filtered by department)
// ==============================
app.get('/api/department-head/expenditures', (req, res) => {
 if (!req.session.user || req.session.user.role !== 'Department Head') {
   return res.status(403).send('Access denied. Department Heads only.');
 }

 getDeptHeadDepartmentId(req, (err, departmentId) => {
   if (err) return res.status(500).send('Error resolving department.');

   const query = `
     SELECT expenditure_id, expenditure_name, expenditure_amount, date
     FROM expenditures
     WHERE department_id = ?
     ORDER BY date DESC
```

```
    `;

    db.query(query, [departmentId], (qErr, results) => {
      if (qErr) return res.status(500).send('Error fetching expenditures.');
      res.json(results || []);
    });
  });
});

// ==============================
// API: Add expenditure
// ==============================


// Configure transporter (use your Gmail + App Password here)


// ==============================
// API: Add expenditure + Notify Admins
// ==============================
app.post('/api/department-head/expenditures/add', (req, res) => {
  if (!req.session.user || req.session.user.role !== 'Department Head') {
    return res.status(403).send('Access denied. Department Heads only.');
  }

  const { expenditure_name, expenditure_amount, date } = req.body;
  if (!expenditure_name || !expenditure_amount || !date) {
    return res.status(400).send('Missing required fields.');
  }

  getDeptHeadDepartmentId(req, (err, departmentId) => {
    if (err) return res.status(500).send('Error resolving department.');

    const insertQuery = `
      INSERT INTO expenditures (department_id, expenditure_name, expenditure_amount,
date)
      VALUES (?, ?, ?, ?)
    `;
    db.query(insertQuery, [departmentId, expenditure_name, expenditure_amount, date],
(qErr) => {
      if (qErr) return res.status(500).send('Error adding expenditure.');
```

```
    // Fetch department name for email
    const deptQuery = `SELECT department_name FROM departments WHERE department_id =
? LIMIT 1`;
    db.query(deptQuery, [departmentId], (deptErr, deptResults) => {
      if (deptErr || !deptResults.length) return res.sendStatus(200);
      const departmentName = deptResults[0].department_name;

      // Fetch all admins
      const adminQuery = `SELECT username, email FROM users WHERE role = 'admin'`;
      db.query(adminQuery, (adminErr, admins) => {
        if (adminErr) return res.sendStatus(200);

        admins.forEach(admin => {
          const mailOptions = {
            from: 'blissfulbarfi@gmail.com',
            to: admin.email,
            subject: 'New Expenditure Logged',
            text: `Hello ${admin.username},

There has been an expenditure by a department head.

Department: ${departmentName}
Requested Amount: ${expenditure_amount}
Purpose : ${expenditure_name}
Submission Date: ${new Date(date).toLocaleDateString()}

You may coordinate with the department head concerned if there are any issues.

Regards,
Your Budget Tracking System`
          };

          transporter.sendMail(mailOptions, (err, info) => {
            if (err) console.error('Error sending email:', err);
            else console.log('Email sent:', info.response);
          });
        });

        res.sendStatus(200); // everything done
      });
    });
  });
```

```
 });
});



// ==============================
// API: Delete expenditure
// ==============================
app.post('/api/department-head/expenditures/delete', (req, res) => {
 if (!req.session.user || req.session.user.role !== 'Department Head') {
   return res.status(403).send('Access denied. Department Heads only.');
 }

 const { id } = req.body;
 if (!id) return res.status(400).send('Missing expenditure id.');

 getDeptHeadDepartmentId(req, (err, departmentId) => {
   if (err) return res.status(500).send('Error resolving department.');

   const query = `
     DELETE FROM expenditures
     WHERE expenditure_id = ? AND department_id = ?
   `;
   db.query(query, [id, departmentId], (qErr, result) => {
     if (qErr) return res.status(500).send('Error deleting expenditure.');
     if (result.affectedRows === 0) {
       return res.status(403).send('Delete denied or expenditure not in your
department.');
     }
     res.sendStatus(200);
   });
 });
});

//delete if smthg goes wrong (2/09/25)
// ==============================
// API: Department Head - Update Budget Request
// ==============================
// ==============================
// API: Department Head - Update Budget Request
// ==============================
// app.post('/api/department-head/budget/update', (req, res) => {
//   if (!req.session.user || req.session.user.role !== 'Department Head') {
```

```
//     return res.status(403).send('Access denied. Department Heads only.');
//   }

//   const { department_id, requested_budget } = req.body;
//   const userId = req.session.user.id;

//   console.log("Incoming update request:", { department_id, requested_budget, userId
});

//   // Step 1: Get the department_id for this dept head user
//   const deptQuery = `
//     SELECT department_id
//     FROM users
//     WHERE id = ? LIMIT 1
//   `;

//   db.query(deptQuery, [userId], (deptErr, deptRows) => {
//     if (deptErr) {
//       console.error("Dept lookup error:", deptErr);
//       return res.status(500).send('Error resolving department.');
//     }
//     if (!deptRows || deptRows.length === 0) {
//       console.log("No department found for this user");
//       return res.status(404).send('Department not found.');
//     }

//     const userDeptId = deptRows[0].department_id;
//     console.log("User belongs to department:", userDeptId);

//     // Step 2: Prevent tampering — only allow their own dept
//     if (Number(userDeptId) !== Number(department_id)) {
//       console.log("Department mismatch! Tried to edit:", department_id, "but user
belongs to:", userDeptId);
//       return res.status(403).send('You can only edit your own department.');
//     }

//     // Step 3: Update requested_budget
//     const updateQuery = `UPDATE departments SET requested_budget = ? WHERE
department_id = ?`;
//     db.query(updateQuery, [requested_budget, department_id], (updateErr, result) =>
{
//       if (updateErr) {
```

```
//         console.error("Update error:", updateErr);
//         return res.status(500).send('Error updating budget.');
//       }
//       console.log("Update result:", result);

//       if (result.affectedRows === 0) {
//         return res.status(404).send('Department not found or update failed.');
//       }

//       res.sendStatus(200);
//     });
//   });
// });

app.post('/api/department-head/budget/update', (req, res) => {
 if (!req.session.user || req.session.user.role !== 'Department Head') {
   return res.status(403).send('Access denied. Department Heads only.');
 }

 const { requested_budget } = req.body;
 const userId = req.session.user.id;

 getDeptHeadDepartmentId(req, (err, departmentId) => {
   if (err) return res.status(500).send('Error resolving department.');

   const updateQuery = `UPDATE departments SET requested_budget = ? WHERE
department_id = ?`;
   db.query(updateQuery, [requested_budget, departmentId], (updateErr, result) => {
     if (updateErr) {
       console.error("Update error:", updateErr);
       return res.status(500).send('Error updating budget.');
     }

     if (result.affectedRows === 0) {
       return res.status(404).send('Department not found or update failed.');
     }

     res.sendStatus(200);
   });
 });
});
```

```
//GENERAL EMPLOYEE ROUTES - EMPLOYEE DASHBOARD

// ==============================
// Employee Dashboard Page
// ==============================
app.get('/employee-dashboard', (req, res) => {
 if (!req.session.user || req.session.user.role !== 'General Employee') {
   return res.status(403).send('Access denied. Employees only.');
 }
 res.sendFile(path.join(__dirname, 'public', 'employee-dashboard.html'));
});

// ==============================
// API: Employee Dashboard Data
// ==============================
app.get('/api/employee-dashboard-data', (req, res) => {
 if (!req.session.user || req.session.user.role !== 'General Employee') {
   return res.status(403).send('Access denied. Employees only.');
 }

 // Step 1: find department_id for this user
 const userId = req.session.user.id;
 const deptIdQuery = `SELECT department_id FROM users WHERE id = ? LIMIT 1`;

 db.query(deptIdQuery, [userId], (err, deptResult) => {
   if (err) return res.status(500).send('Error resolving department.');
   if (!deptResult || deptResult.length === 0) return res.status(404).send('Department
not found.');

   const departmentId = deptResult[0].department_id;

   // Step 2: get department budget info
  const departmentQuery = `
 SELECT d.department_id, d.department_name, d.requested_budget, d.admin_comments,
d.budget_status,
        u.username AS department_head
 FROM departments d
 LEFT JOIN users u ON d.department_head_id = u.id
```

```
 WHERE d.department_id = ?
 LIMIT 1
`;


   db.query(departmentQuery, [departmentId], (deptErr, deptRows) => {
     if (deptErr) return res.status(500).send('Error fetching department data.');
     if (!deptRows || deptRows.length === 0) return res.status(404).send('No
department data found.');

     const departmentData = deptRows[0];

     // Step 3: get expenditures for that department
     const expendituresQuery = `
       SELECT SUM(expenditure_amount) AS actual_spent
       FROM expenditures
       WHERE department_id = ?
     `;

     db.query(expendituresQuery, [departmentId], (expErr, expRows) => {
       if (expErr) return res.status(500).send('Error fetching expenditures.');

       const actualSpent = expRows && expRows[0] ? expRows[0].actual_spent || 0 : 0;

       res.json({
         department: departmentData,
         spending: {
           department_name: departmentData.department_name,
           requested_budget: departmentData.requested_budget,
           actual_spent: actualSpent
         }
       });
     });
   });
 });
});


// //Values you want to insert
// const username = 'user2';
// const plainPassword = 'newUserPass123';
// const role = 'General Employee';
```

```
// const email = 'sneha.u.gautam@gmail.com';
// const departmentId = 2;

// //FOR WHEN U WANNA INSERT A NEW USER - JUST REPLACE VALUES

// bcrypt.hash(plainPassword, 10, (err, hash) => {
//   if (err) {
//     console.error('Error hashing password:', err);
//     return;
//   }

//   const query = `
//     INSERT INTO users (username, password, role, email, department_id)
//     VALUES (?, ?, ?, ?, ?)
//   `;

//   db.query(query, [username, hash, role, email, departmentId], (qErr, result) => {
//     if (qErr) {
//       console.error('Error inserting user:', qErr);
//     } else {
//       console.log('New user inserted with ID:', result.insertId);
//     }
//   });
// });

// ==============================
// API: Department Head Budget Status
// ==============================
app.get('/api/department-head/budget-status', (req, res) => {
 if (!req.session.user || req.session.user.role !== 'Department Head') {
   return res.status(403).send('Access denied. Department Heads only.');
 }

 const userId = req.session.user.id;

 // Step 1: Find department_id for this department head
 const deptQuery = `
   SELECT d.department_id, d.department_name, d.requested_budget,
          d.budget_status, d.admin_comments, u.username AS department_head
   FROM departments d
   JOIN users u ON d.department_head_id = u.id
   WHERE u.id = ?
```

```
  `;

  db.query(deptQuery, [userId], (err, rows) => {
    if (err) {
      console.error("Error fetching budget status:", err);
      return res.status(500).send('Error fetching budget status.');
    }
    res.json(rows); // will be [] if no match
  });
});




// Start Server
app.listen(3000, () => {
    console.log('Server running on http://localhost:3000');
});
```

Application Dependencies : (define project dependencies and scripts required to run the application server)

package.json

```
{
  "name": "login-project",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": "",
  "dependencies": {
    "bcrypt": "^5.1.1",
    "body-parser": "^1.20.3",
    "canvas": "^3.1.0",
    "chart.js": "^4.4.9",
```

```
    "crypto": "^1.0.1",
    "dotenv": "^17.2.1",
    "express": "^4.21.2",
    "express-mysql-session": "^3.0.3",
    "express-session": "^1.18.1",
    "jspdf": "^3.0.1",
    "mysql": "^2.18.1",
    "mysql2": "^3.13.0",
    "nodemailer": "^7.0.10",
    "pdfkit": "^0.17.1"
  }
}
```

User-Side Codes

Applicable to all users of the web application

Login Page

index.html (Provides a common login interface for all users and routes them based on assigned roles.)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <div class="login-container">budget tracking system
        <h2>Login</h2>
        <form action="/login" method="POST">
            <label for="username">LoginName:</label>
            <input type="text" id="username" name="username" required><br><br>

            <label for="password">Password:</label>
            <input type="password" id="password" name="password" required><br><br>

            <button type="submit">Login</button>
        </form>
```

```
      <!-- Forgot Password Link -->
      <p><a href="request-reset.html">Forgot Password?</a></p>
   </div>
</body>
</html>
```

request-reset.html (Provides an option to reset password, directing the user to another page)

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8" />
 <title>Request Password Reset</title>
 <link rel="stylesheet" href="styles.css" />
</head>
<body>
 <div class="reset-container">
   <h2>Forgot Password?</h2>
   <form action="/request-reset" method="POST">
     <label for="username">Enter Your Username:</label>
     <input type="text" id="username" name="username" required><br><br>
     <button type="submit">Send Verification Code</button>
   </form>
 </div>
</body>
</html>
```

reset-password.html (page to reset password with authentication code, etc.)

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8" />
 <title>Reset Password</title>
 <link rel="stylesheet" href="styles.css" />
</head>
<body>
 <div class="reset-container">
   <h2>Reset Your Password</h2>
   <form action="/reset-password" method="POST">
     <label for="newPassword">New Password:</label>
```

```
    <input type="password" name="newPassword" id="newPassword" required><br><br>

    <label for="confirmPassword">Confirm New Password:</label>
    <input type="password" name="confirmPassword" id="confirmPassword"
required><br><br>

    <button type="submit">Set New Password</button>
   </form>
 </div>
</body>
</html>
```

verify-reset-code.html (verifies that the emailed reset password code the user enters in the web application matches the one sent.)

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8" />
 <title>Verify Code</title>
 <link rel="stylesheet" href="styles.css" />
</head>
<body>
 <div class="reset-container">
   <h2>Enter Verification Code</h2>
   <form action="/verify-code" method="POST">
     <label for="code">Enter the 5-digit code sent to your email:</label>
     <input type="text" id="code" name="code" required><br><br>
     <button type="submit">Verify Code</button>
   </form>
 </div>
</body>
</html>
```

Role-Based Dashboards

Admin Interface

admin.html

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta charset="UTF-8">
    <title>Admin Panel</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body style="background-color: #54A0FF; color: black; font-family: Arial;">

    <h2 style="text-align:center;">Admin Functions</h2>

    <div class="admin-container">
        <button onclick="location.href='/admin-dashboard'">ADMIN DASHBOARD</button>
        <button onclick="location.href='/budget-requests'">BUDGET REQUESTS AND
APPROVALS</button>
        <button onclick="location.href='/user-management'">USER MANAGEMENT</button>
        <button onclick="location.href='/budget-reports'">BUDGET REPORTS</button>
    </div>

</body>
</html>
```

admin-dashboard.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Admin Dashboard</title>
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script> <!-- Chart.js for the
graph -->
    <link rel="stylesheet" href="styles.css">
</head>
<body style="background-color: #a2c2f4; color: black; font-family: Arial,
sans-serif;">

    <!-- Page Heading -->
    <h1 style="text-align: center; font-size: 36px; margin-top: 20px;">WELCOME TO THE
ADMIN DASHBOARD</h1>

    <!-- Department Table -->
    <h3 style="text-align: center; margin-top: 30px;">Department Budget
Information</h3>
```

```
   <table border="1" cellpadding="10" style="width: 80%; margin: 0 auto;
border-collapse: collapse;">
       <thead style="background-color: #4b7bec; color: white;">
           <tr>
               <th style="padding: 12px;">Department</th>
               <th style="padding: 12px;">Department Head</th>
               <th style="padding: 12px;">Amount</th>
               <th style="padding: 12px;">Comments</th>
           </tr>
       </thead>
       <tbody id="department-table">
           <!-- Table rows will be populated dynamically with server data -->
       </tbody>
   </table>

   <!-- Graph: Planned vs Actual -->
   <h3 style="text-align: center; margin-top: 40px;">Planned vs Actual Spending</h3>
   <canvas id="spendingChart" width="400" height="200" style="margin: 0 auto; display:
block;"></canvas>

   <!-- Real-time Spending Updates Button -->
   <div style="text-align: center; margin-top: 30px;">
       <button onclick="window.location.href='/real-time-updates'" style="padding:
10px 20px; background-color: #4b7bec; color: white; border: none; border-radius: 5px;
cursor: pointer;">
           Email Me Real-Time Spending Updates
       </button>
   </div>

   <script>
       // Function to fetch data for the table and graph from the server
       // fetch('/admin-dashboard-data')
       //     .then(response => response.json())
       //     .then(data => {
       //         // Populate table with department data
       //         const tableBody = document.getElementById('department-table');
       //         data.departments.forEach(department => {
       //             const row = document.createElement('tr');
       //             row.innerHTML = `
       //                 <td style="padding:
12px;">${department.department_name}</td>
```

```
//                 <td style="padding:
12px;">${department.department_head}</td>
//                 <td style="padding:
12px;">${department.requested_budget}</td>
//                 <td style="padding: 12px;">${department.admin_comments ||
'No comments'}</td>
//             `;
//             tableBody.appendChild(row);
//         });

//         // Prepare data for the graph
//         const labels = data.departments.map(dep => dep.department_name);
//         const plannedData = data.departments.map(dep =>
dep.requested_budget);
//         const actualData = data.expenditures.map(exp => exp.actual_spent);

//         // Create the graph using Chart.js
//         const ctx =
document.getElementById('spendingChart').getContext('2d');
//         new Chart(ctx, {
//             type: 'bar',
//             data: {
//                 labels: labels,
//                 datasets: [
//                     {
//                         label: 'Planned Spending',
//                         data: plannedData,
//                         backgroundColor: '#FF6384',
//                         borderColor: '#FF6384',
//                         borderWidth: 1
//                     },
//                     {
//                         label: 'Actual Spending',
//                         data: actualData,
//                         backgroundColor: '#36A2EB',
//                         borderColor: '#36A2EB',
//                         borderWidth: 1
//                     }
//                 ]
//             },
//             options: {
//                 scales: {
```

```
      //                      y: {
      //                          beginAtZero: true
      //                      }
      //                  }
      //              }
      //          });
      //      });

      fetch('/admin-dashboard-data')
.then(response => response.json())
.then(data => {
  // Populate table
  const tableBody = document.getElementById('department-table');
  data.forEach(department => {
    const row = document.createElement('tr');
    row.innerHTML = `
      <td style="padding: 12px;">${department.department_name}</td>
      <td style="padding: 12px;">${department.department_head}</td>
      <td style="padding: 12px;">${department.requested_budget}</td>
      <td style="padding: 12px;">${department.admin_comments || 'No comments'}</td>
    `;
    tableBody.appendChild(row);
  });

  // Graph data
  const labels = data.map(dep => dep.department_name);
  const plannedData = data.map(dep => dep.requested_budget);
  const actualData = data.map(dep => dep.actual_spent);

  // Chart
  const ctx = document.getElementById('spendingChart').getContext('2d');
  new Chart(ctx, {
    type: 'bar',
    data: {
      labels: labels,
      datasets: [
        {
          label: 'Planned Spending',
          data: plannedData,
          backgroundColor: '#FF6384',
          borderColor: '#FF6384',
          borderWidth: 1
```

```
        },
        {
          label: 'Actual Spending',
          data: actualData,
          backgroundColor: '#36A2EB',
          borderColor: '#36A2EB',
          borderWidth: 1
        }
      ]
    },
    options: {
      scales: {
        y: { beginAtZero: true }
      }
    }
  });
});

  </script>
</body>
</html>
```

budget-requests.html

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <title>Budget Requests and Approvals</title>
 <link rel="stylesheet" href="styles.css">
</head>
<p style="text-align:center; font-size:14px; color:#333;">
 <b>Status codes:</b> A = Approved, D = Denied, P = Pending
</p>

<body style="background-color: #a2c2f4; font-family: Arial, sans-serif;">

 <h1 style="text-align: center; font-size: 32px; margin-top: 20px;">budget requests
and approvals</h1>

 <table border="1" cellpadding="10" style="width: 80%; margin: 20px auto;
border-collapse: collapse; text-align: center;">
```

```
  <thead style="background-color: #4b7bec; color: white;">
    <tr>
      <th>Department</th>
      <th>Department Head</th>
      <th>Amount</th>
      <th>Approve/Deny</th>
      <th>Comments</th>
    </tr>
  </thead>
  <tbody id="budget-table"></tbody>
</table>

<script>
  // Fetch and display department data
  // fetch('/admin-dashboard-data')
  //   .then(res => res.json())
  //   .then(data => {
  //     const table = document.getElementById('budget-table');
  //     data.departments.forEach(row => {
  //       const tr = document.createElement('tr');

  //       // Format budget status
  //       const statusCell = row.budget_status
  //         ? `${row.budget_status} <br><button
onclick="editStatus('${row.department_id}', '${row.budget_status}')">Edit
status?</button>`
  //         : `<button onclick="editStatus('${row.department_id}', '')">Add
status?</button>`;

  //       // Format admin comment
  //       const commentCell = row.admin_comments
  //         ? `${row.admin_comments} <br><button
onclick="editComment('${row.department_id}', \`${row.admin_comments}\`)">Edit
comment?</button>`
  //         : `<button onclick="editComment('${row.department_id}', '')">Add
comment?</button>`;

  //       tr.innerHTML = `
  //         <td>${row.department_name}</td>
  //         <td>${row.department_head}</td>
  //         <td>$${row.requested_budget}</td>
  //         <td>${statusCell}</td>
```

```
  //          <td>${commentCell}</td>
  //        `;
  //        table.appendChild(tr);
  //      });
  //    });
  fetch('/admin-dashboard-data')
.then(res => res.json())
.then(data => {
  const table = document.getElementById('budget-table');
  data.forEach(row => {    // ✅ data is the array directly now
    const tr = document.createElement('tr');

    // Format budget status
    const statusCell = row.budget_status
      ? `${row.budget_status} <br><button onclick="editStatus('${row.department_id}',
'${row.budget_status}')">Edit status?</button>`
      : `<button onclick="editStatus('${row.department_id}', '')">Add
status?</button>`;

    // Format admin comment
    const commentCell = row.admin_comments
      ? `${row.admin_comments} <br><button
onclick="editComment('${row.department_id}', \`${row.admin_comments}\`)">Edit
comment?</button>`
      : `<button onclick="editComment('${row.department_id}', '')">Add
comment?</button>`;

    tr.innerHTML = `
      <td>${row.department_name}</td>
      <td>${row.department_head}</td>
      <td>₹${row.requested_budget}</td>
      <td>${statusCell}</td>
      <td>${commentCell}</td>
    `;
    table.appendChild(tr);
  });
});


  // Add or edit budget status
 function editStatus(deptId, currentStatus) {
// Show user-friendly current status (map DB letters to words)
```

```
 const mapLetterToWord = { A: 'Approved', D: 'Denied', P: 'Pending' };
 const currentWord = mapLetterToWord[currentStatus] || currentStatus || '';

 const raw = prompt(
   `Enter new status for department ${deptId}:\nUse A / D / P or Approved / Denied /
Pending`,
   currentWord
 );

 if (raw === null) return; // user cancelled

 const input = raw.trim().toLowerCase();
 if (!input) return alert('No status entered.');

 // Normalize input to single-letter codes
 let newStatusLetter = null;
 if (input === 'a' || input === 'approved') newStatusLetter = 'A';
 else if (input === 'd' || input === 'denied') newStatusLetter = 'D';
 else if (input === 'p' || input === 'pending') newStatusLetter = 'P';
 else return alert('Invalid status. Use A, D, P or Approved, Denied, Pending.');

 // optional: confirm
 if (!confirm(`Change status for dept ${deptId} to "${newStatusLetter}"?`)) return;

 // send to server
 fetch('/update-status', {
   method: 'POST',
   headers: { 'Content-Type': 'application/json' },
   body: JSON.stringify({ department_id: deptId, status: newStatusLetter })
 })
 .then(async res => {
   if (!res.ok) {
     const text = await res.text().catch(() => '');
     throw new Error(text || `Status update failed (HTTP ${res.status})`);
   }
   alert('Status updated successfully!');
   // Ideally re-fetch the table instead of reload; reload is simplest
   location.reload();
 })
 .catch(err => {
   console.error('Status update error:', err);
   alert('Error updating status: ' + err.message);
```

```
 });
}



  // Add or edit admin comment
  function editComment(deptId, currentComment) {
 const newComment = prompt('Enter your comment:', currentComment || '');
 if (newComment === null) return; // cancelled

 const trimmed = newComment.trim();
 if (trimmed === '') return alert('Comment cannot be empty.');

 // send to server
 fetch('/update-comment', {
   method: 'POST',
   headers: { 'Content-Type': 'application/json' },
   body: JSON.stringify({ department_id: deptId, comment: trimmed })
 })
 .then(async res => {
   if (!res.ok) {
     const text = await res.text().catch(() => '');
     throw new Error(text || `Comment update failed (HTTP ${res.status})`);
   }
   alert('Comment updated successfully!');
   location.reload();
 })
 .catch(err => {
   console.error('Comment update error:', err);
   alert('Error updating comment: ' + err.message);
 });
}
 </script>
</body>
</html>
```

user-management.html

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
```

```
 <title>User Management</title>
</head>
<body style="background-color: #a2c2f4; font-family: Arial, sans-serif;">

 <h1 style="text-align: center; font-size: 32px; margin-top: 20px;">user
management</h1>

 <div style="width: 80%; margin: 20px auto; display: flex; justify-content:
space-between; flex-wrap: wrap; gap: 10px;">
 <label>
   <div style="width: 80%; margin: 10px auto; text-align: right;">
 <button onclick="addUser()" style="padding: 8px 16px; background-color: #4b7bec;
color: white; border: none; border-radius: 5px; cursor: pointer;">
   ➕ Add User
 </button>
</div>

   Filter by Department:
   <select id="department-filter">
     <option value="all">All Departments</option>
   </select>
 </label>
 <label>
   Filter by Role:
   <select id="role-filter">
     <option value="all">All Roles</option>
     <option value="admin">admin</option>
     <option value="General Employee">General Employee</option>
     <option value="Department Head">Department Head</option>
   </select>
 </label>
 <label>
   Sort by Name:
   <select id="sort-filter" onchange="applyFilters()">
     <option value="asc">A to Z</option>
     <option value="desc">Z to A</option>
   </select>
 </label>
 <label>
   Search:
   <input type="text" id="search-input" placeholder="Search by name, role, or
department" oninput="applyFilters()">
```

```
 </label>
 <button onclick="applyFilters()">Apply Filters</button>
</div>


 <table border="1" cellpadding="10" style="width: 80%; margin: 0 auto;
border-collapse: collapse; text-align: center;">
   <thead style="background-color: #4b7bec; color: white;">
     <tr>
       <th onclick="toggleSort()">Username ⬍</th>
       <th>Role</th>
       <th>Department</th>
       <th>Actions</th>
     </tr>
   </thead>
   <tbody id="users-table"></tbody>
 </table>

 <script>
   const departmentOptions = ['Marketing', 'Finance', 'HR', 'IT', 'Test Department'];
const roleOptions = ['Admin', 'Department Head', 'General Employee'];

   let users = [];
   let sortAsc = true;

   async function fetchUsers() {
     try {
       const res = await fetch('/api/users');
       const data = await res.json();
       users = data;
       populateDepartmentFilter(data);
       renderTable(data);
     } catch (err) {
       console.error('Error fetching users:', err);
     }
   }

   function populateDepartmentFilter(data) {
     const select = document.getElementById('department-filter');
     const departments = [...new Set(data.map(u =>
u.department_name).filter(Boolean))];
     select.innerHTML = '<option value="all">All Departments</option>';
```

```
    departments.forEach(dept => {
      const option = document.createElement('option');
      option.value = dept;
      option.textContent = dept;
      select.appendChild(option);
    });
  }

  function renderTable(data) {
    const table = document.getElementById('users-table');
    table.innerHTML = '';
    data.forEach(user => {
      const tr = document.createElement('tr');
      tr.innerHTML = `
        <td>${user.username}</td>
        <td>${user.role}</td>
        <td>${user.department_name || '—'}</td>
        <td>
          <button onclick="editUser(${user.id}, '${user.username}', '${user.role}',
'${user.department_name || ''}')">Edit</button>
          <button onclick="deleteUser(${user.id})">Delete</button>
        </td>
      `;
      table.appendChild(tr);
    });
  }

function applyFilters() {
const dept = document.getElementById('department-filter').value;
const role = document.getElementById('role-filter').value;
const sort = document.getElementById('sort-filter').value;
const searchTerm = document.getElementById('search-input').value.toLowerCase();

let filtered = [...users];

if (dept !== 'all') {
  filtered = filtered.filter(u => u.department_name === dept);
}
if (role !== 'all') {
  filtered = filtered.filter(u => u.role === role);
}
```

```
 // New: Search filter
 if (searchTerm.trim() !== '') {
   filtered = filtered.filter(u =>
     u.username.toLowerCase().includes(searchTerm) ||
     (u.role && u.role.toLowerCase().includes(searchTerm)) ||
     (u.department_name && u.department_name.toLowerCase().includes(searchTerm))
   );
 }

 filtered.sort((a, b) =>
   sort === 'asc'
     ? a.username.localeCompare(b.username)
     : b.username.localeCompare(a.username)
 );

 renderTable(filtered);
}




   function toggleSort() {
     users.sort((a, b) => {
       return sortAsc
         ? a.username.localeCompare(b.username)
         : b.username.localeCompare(a.username);
     });
     sortAsc = !sortAsc;
     applyFilters();
   }

   function editUser(id, username, role, departmentName) {
 // Username edit stays text-based
 const newUsername = prompt('Edit username:', username);
 if (!newUsername) return;

 // Create dropdowns
 const newRole = promptDropdown('Select role:', roleOptions, role);
 const newDepartment = promptDropdown('Select department:', departmentOptions,
departmentName);

 if (!newRole || !newDepartment) {
   alert('You must select both role and department.');
```

```
    return;
  }

  fetch('/api/users/update', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ id, username: newUsername, role: newRole, department_name:
newDepartment })
  }).then(() => fetchUsers());
}

// Helper to show dropdown via prompt-like approach
function promptDropdown(label, options, current) {
  const display = options.map((opt, i) => `${i + 1}. ${opt}`).join('\n');
  const choice = prompt(`${label}\n${display}\n(Current: ${current})`);
  const index = parseInt(choice);
  return isNaN(index) || index < 1 || index > options.length ? null : options[index -
1];
}

    function deleteUser(id) {
      if (!confirm('Are you sure you want to delete this user?')) return;

      fetch('/api/users/delete', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ id })
      }).then(() => fetchUsers());
    }
async function addUser() {
  const username = prompt('Enter username:');
  const email = prompt('Enter email:');
  const password = prompt('Enter password:'); // will be hashed in backend

  if (!username || !email || !password) {
    alert('Username, email, and password are required.');
    return;
  }

  // Dropdowns for role and department
  const role = promptDropdown('Select role:', roleOptions, '');
  const department = promptDropdown('Select department:', departmentOptions, '');
```

```
 if (!role || !department) {
   alert('You must select both role and department.');
   return;
 }

 const newUser = { username, email, password, role, department_name: department };

 try {
   const res = await fetch('/api/users/add', {
     method: 'POST',
     headers: { 'Content-Type': 'application/json' },
     body: JSON.stringify(newUser)
   });

   if (res.ok) {
     alert('✅ User added successfully!');
     fetchUsers(); // refresh table
   } else {
     alert('❌ Failed to add user.');
   }
 } catch (err) {
   console.error('Error adding user:', err);
   alert('An error occurred while adding the user.');
 }
}

   fetchUsers();
 </script>
</body>
</html>
```

budget-reports.html

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <title>Budget Reports</title>
 <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
 <script src="https://cdn.jsdelivr.net/npm/chartjs-plugin-datalabels@2"></script>
```

```
 <script
src="https://cdnjs.cloudflare.com/ajax/libs/jspdf/2.5.1/jspdf.umd.min.js"></script>
 <script
src="https://cdnjs.cloudflare.com/ajax/libs/html2canvas/1.4.1/html2canvas.min.js"></sc
ript>
 <link rel="stylesheet" href="styles.css">
</head>
<body style="background-color: #a2c2f4; font-family: Arial, sans-serif;">

 <h1 style="text-align: center; font-size: 32px; margin-top: 20px;">budget
reports</h1>

 <table border="1" cellpadding="10" style="width: 90%; margin: 20px auto;
border-collapse: collapse; text-align: center;">
   <thead style="background-color: #4b7bec; color: white;">
     <tr>
       <th>Department</th>
       <th>Department Head</th>
       <th>Amount</th>
       <th>Comments</th>
       <th>Email to Me</th>
       <th>Download PDF</th>
     </tr>
   </thead>
   <tbody id="report-table" style="background-color: white;"></tbody>
 </table>

 <h2 style="text-align: center; font-size: 24px;">Overall Department Budget Share</h2>
 <div style="display: flex; justify-content: center;">
   <canvas id="budgetPieChart" style="max-width: 600px; background-color: white;
padding: 20px; border-radius: 10px;"></canvas>
 </div>

 <script>
   let pdfChartInstance = null;

   fetch('/api/budget-reports-data')
     .then(res => res.json())
     .then(data => {
       const table = document.getElementById('report-table');
       let labels = [], values = [];
```

```
      data.forEach(row => {
        labels.push(row.department_name);
        values.push(Number(row.requested_budget.toString().replace(/[^0-9.]/g, '')));

        const tr = document.createElement('tr');
        tr.innerHTML = `
          <td>${row.department_name}</td>
          <td>${row.department_head}</td>
          <td>₹${row.requested_budget}</td>
          <td>${row.admin_comments || '—'}</td>
          <td><button onclick='emailPDFToUser(${JSON.stringify(row)})'>Email to
Me</button></td>
          <td><button onclick='generatePDF(${JSON.stringify(row)})'>Download
PDF</button></td>
        `;
        table.appendChild(tr);
      });

      // Save for chart rendering
      window.chartLabels = labels;
      window.chartValues = values;

      const ctx = document.getElementById('budgetPieChart').getContext('2d');
      new Chart(ctx, {
        type: 'pie',
        data: {
          labels,
          datasets: [{
            label: 'Budget Share',
            data: values,
            backgroundColor: [
              '#FF6384', '#36A2EB', '#FFCE56', '#66BB6A',
              '#BA68C8', '#FFA07A', '#90CAF9', '#F06292'
            ],
            borderColor: '#ffffff',
            borderWidth: 2
          }]
        },
        options: {
          plugins: {
            legend: { position: 'right' },
            datalabels: {
```

```
              color: '#fff',
              font: { weight: 'bold', size: 14 },
              formatter: (value, ctx) => {
                const label = ctx.chart.data.labels[ctx.dataIndex];
                const total = ctx.chart.data.datasets[0].data.reduce((a, b) => a + b,
0);
                return `${label}\n${((value / total) * 100).toFixed(1)}%`;
              }
            }
          }
        },
        plugins: [ChartDataLabels]
      });
    });

  //REPLACE IF SOMETHING GOES WRONG IN THE CURRENT CODE
    // function generatePDF(row) {
  //   const content = document.getElementById('pdf-content');
  //   html2canvas(content, { useCORS: true, backgroundColor: "#ffffff"
}).then(canvas => {
  //     const imgData = canvas.toDataURL('image/png');
  //     const { jsPDF } = window.jspdf;
  //     const pdf = new jsPDF('p', 'pt', 'a4');
  //     const width = pdf.internal.pageSize.getWidth();
  //     const height = (canvas.height * width) / canvas.width;
  //     pdf.addImage(imgData, 'PNG', 20, 20, width - 40, height - 60);
  //     pdf.save(`${row.department_name}_Budget_Report.pdf`);
  //   });
  // }

  async function generatePDF(row) {
 // 1. Populate text fields
 document.getElementById('pdf-dept-name').textContent = row.department_name;
 document.getElementById('pdf-department-name').textContent = row.department_name;
 document.getElementById('pdf-department-head').textContent = row.department_head;
 document.getElementById('pdf-requested-amount').textContent = row.requested_budget;
 document.getElementById('pdf-budget-status').textContent = row.budget_status || '—';
 document.getElementById('pdf-admin-comments').textContent = row.admin_comments ||
'—';
 document.getElementById('pdf-email').textContent = "contact@" +
row.department_name.toLowerCase() + ".com"; // example
```

```
// 2. Re-render pie chart inside the PDF template
const chartCanvas = document.getElementById("pdf-chart").getContext("2d");
if (pdfChartInstance) {
  pdfChartInstance.destroy();
}
pdfChartInstance = new Chart(chartCanvas, {
  type: 'pie',
  data: {
    labels: window.chartLabels,
    datasets: [{
      label: 'Budget Share',
      data: window.chartValues,
      backgroundColor: [
        '#FF6384', '#36A2EB', '#FFCE56', '#66BB6A',
        '#BA68C8', '#FFA07A', '#90CAF9', '#F06292'
      ],
      borderColor: '#ffffff',
      borderWidth: 2
    }]
  },
  options: {
    plugins: {
      legend: { position: 'right' },
      datalabels: {
        color: '#fff',
        font: { weight: 'bold', size: 14 },
        formatter: (value, ctx) => {
          const label = ctx.chart.data.labels[ctx.dataIndex];
          const total = ctx.chart.data.datasets[0].data.reduce((a, b) => a + b, 0);
          return `${label}\n${((value / total) * 100).toFixed(1)}%`;
        }
      }
    }
  },
  plugins: [ChartDataLabels]
});

// 3. Wait a tick so chart renders
await new Promise(resolve => setTimeout(resolve, 500));

// 4. Convert hidden DOM → canvas → PDF
const content = document.getElementById('pdf-content');
```

```
 html2canvas(content, { useCORS: true, backgroundColor: "#ffffff" }).then(canvas => {
   const imgData = canvas.toDataURL('image/png');
   const { jsPDF } = window.jspdf;
   const pdf = new jsPDF('p', 'pt', 'a4');
   const width = pdf.internal.pageSize.getWidth();
   const height = (canvas.height * width) / canvas.width;
   pdf.addImage(imgData, 'PNG', 20, 20, width - 40, height - 60);
   pdf.save(`${row.department_name}_Budget_Report.pdf`);
 });
}



   // ✅ FIXED: This now just sends department data to backend.
  async function emailPDFToUser(row) {
 try {
   // 1. Populate PDF content (reuse same logic as generatePDF)
   document.getElementById('pdf-dept-name').textContent = row.department_name;
   document.getElementById('pdf-department-name').textContent = row.department_name;
   document.getElementById('pdf-department-head').textContent = row.department_head;
   document.getElementById('pdf-requested-amount').textContent = row.requested_budget;
   document.getElementById('pdf-budget-status').textContent = row.budget_status ||
'—';
   document.getElementById('pdf-admin-comments').textContent = row.admin_comments ||
'—';
   document.getElementById('pdf-email').textContent = "contact@" +
row.department_name.toLowerCase() + ".com";

   // 2. Re-render pie chart
   const chartCanvas = document.getElementById("pdf-chart").getContext("2d");
   if (pdfChartInstance) pdfChartInstance.destroy();
   pdfChartInstance = new Chart(chartCanvas, {
     type: 'pie',
     data: {
       labels: window.chartLabels,
       datasets: [{
         data: window.chartValues,
         backgroundColor: [
           '#FF6384', '#36A2EB', '#FFCE56', '#66BB6A',
           '#BA68C8', '#FFA07A', '#90CAF9', '#F06292'
         ],
         borderColor: '#ffffff',
         borderWidth: 2
```

```
      }]
    },
    options: { plugins: { legend: { position: 'right' } } }
  });

  await new Promise(resolve => setTimeout(resolve, 500));

  // 3. Convert to PDF → base64
  const content = document.getElementById('pdf-content');
  const canvas = await html2canvas(content, { useCORS: true, backgroundColor:
"#ffffff" });
  const imgData = canvas.toDataURL('image/png');
  const { jsPDF } = window.jspdf;
  const pdf = new jsPDF('p', 'pt', 'a4');
  const width = pdf.internal.pageSize.getWidth();
  const height = (canvas.height * width) / canvas.width;
  pdf.addImage(imgData, 'PNG', 20, 20, width - 40, height - 60);

  // Get base64 string
  const pdfBase64 = pdf.output('datauristring').split(',')[1];

  // 4. Send to backend
  const response = await fetch('/email-budget-report', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ pdfBase64, department_name: row.department_name })

  });



  const result = await response.text();
  alert(result);
 } catch (err) {
  console.error(err);
  alert("Failed to send email.");
 }
}



  window.generatePDF = generatePDF;
 </script>
```

```
 <!-- Hidden PDF content (for download PDF only, not for email) -->
 <div id="pdf-content" style="position: absolute; left: -9999px; top: -9999px;">
   <div style="padding: 30px; font-family: Arial, sans-serif;">
     <h2 style="color: #3b2fd4;">BUDGET REPORT FOR DEPARTMENT <span
id="pdf-dept-name"></span></h2>
     <div style="background-color: #e9cfcf; padding: 20px; font-size: 16px;">
       <p><b>Department name :</b> <span id="pdf-department-name"></span></p>
       <p><b>Department head :</b> <span id="pdf-department-head"></span></p>
       <p><b>Amount requested :</b> ₹<span id="pdf-requested-amount"></span></p>
       <p><b>Approval status :</b> <span id="pdf-budget-status"></span></p>
       <p><b>Admin comments :</b><br><span id="pdf-admin-comments"></span></p>
       <p><b>Email for queries :</b> <span id="pdf-email"></span></p>
     </div>
     <h3 style="margin-top: 40px;">Departmental income percentage :</h3>
     <canvas id="pdf-chart" width="500" height="320" style="margin-top:
20px;"></canvas>
   </div>
 </div>

</body>
</html>
```

Department-Head Interface

department-head.html

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Department Head Panel</title>
   <link rel="stylesheet" href="styles.css">
</head>
<body style="background-color: #54A0FF; color: black; font-family: Arial;">

   <h2 style="text-align:center;">Admin Functions</h2>

   <div class="dept-head-container" style="display: flex; flex-direction: column;
align-items: center; gap: 15px; margin-top: 30px;">
```

```
      <button onclick="location.href='/department-head-dashboard'">DEPARTMENT
DASHBOARD</button>
      <button
onclick="location.href='/department-head-expenditures'">EXPENDITURES</button>
   </div>

</body>
</html>
```

department-head-dashboard.html

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <title>Department Head Dashboard</title>
</head>
<body style="background-color: #a2c2f4; font-family: Arial, sans-serif;">

 <h1 style="text-align: center; font-size: 32px; margin-top: 20px;">Department Head
Dashboard</h1>

 <!-- Table 1: Budget Approval Status -->
 <h2 style="text-align: center; margin-top: 30px;">BUDGET APPROVAL STATUS</h2>
 <table border="1" cellpadding="10" style="width: 80%; margin: 20px auto;
border-collapse: collapse; text-align: center;">
   <thead style="background-color: #4b7bec; color: white;">
     <tr>
       <th>Department</th>
       <th>Department Head</th>
       <th>Amount</th>
       <th>Approve/Deny</th>
       <th>Comments</th>
       <th>Edit</th>
     </tr>
   </thead>
   <tbody id="budget-status-table"></tbody>
 </table>

 <!-- Table 2: Employee Details -->
 <h2 style="text-align: center; margin-top: 30px;">EMPLOYEE DETAILS</h2>
```

```
<table border="1" cellpadding="10" style="width: 80%; margin: 20px auto;
border-collapse: collapse; text-align: center;">
  <thead style="background-color: #4b7bec; color: white;">
    <tr>
      <th>Employee Name</th>
      <th>Employee Department</th>
      <th>Employee Role</th>
      <th>Employee Email</th>
      <th>Edit</th>
      <th>Remove</th>
    </tr>
  </thead>
  <tbody id="employee-details-table"></tbody>
</table>

<script>
  let budgetData = [];
  let employees = [];

  // Fetch Budget Approval Status (only for this department)
  async function fetchBudgetStatus() {
    try {
      const res = await fetch('/api/department-head/budget-status');
      const data = await res.json();
      budgetData = data;
      renderBudgetStatus(data);
    } catch (err) {
      console.error('Error fetching budget status:', err);
    }
  }

  function renderBudgetStatus(data) {
    const table = document.getElementById('budget-status-table');
    table.innerHTML = '';
    data.forEach(item => {
      const tr = document.createElement('tr');
      tr.innerHTML = `
        <td>${item.department_name}</td>
        <td>${item.department_head}</td>
        <td>₹${Number(item.requested_budget).toLocaleString()}</td>
        <td>${item.budget_status}</td>
        <td>${item.admin_comments || '—'}</td>
```

```
        <td><button onclick="editBudgetAmount(${item.department_id},
${item.requested_budget})">Edit</button></td>
      `;
      table.appendChild(tr);
    });
  }

  //REPLACE IF SOMETHING GOES WRONG 3/09/25
  // function editBudgetAmount(departmentId, currentAmount) {
  //   const newAmount = prompt('Enter new budget amount:', currentAmount);
  //   if (!newAmount || isNaN(newAmount)) return alert('Invalid amount.');

  //   fetch('/api/department-head/budget/update', {
  //     method: 'POST',
  //     headers: { 'Content-Type': 'application/json' },
  //     body: JSON.stringify({ department_id: departmentId, requested_budget:
newAmount })



  //   })
  //   .then(res => {
  //     if (res.ok) {
  //       alert('Budget updated successfully!');
  //       fetchBudgetStatus();
  //     } else {
  //       alert('Failed to update budget.');
  //     }
  //   });
  //   console.log("Budget status query returned:", rows);

  // }

  function editBudgetAmount(departmentId, currentAmount) {
 const newAmount = prompt('Enter new budget amount:', currentAmount);
 if (!newAmount || isNaN(newAmount)) return alert('Invalid amount.');

 fetch('/api/department-head/budget/update', {
   method: 'POST',
   headers: { 'Content-Type': 'application/json' },
   body: JSON.stringify({ requested_budget: newAmount }) // 🚨 remove department_id
 })
 .then(res => {
```

```
    if (res.ok) {
      alert('Budget updated successfully!');
      fetchBudgetStatus();
    } else {
      alert('Failed to update budget.');
    }
  });
}



  // Fetch Employees (only those in this department)
  async function fetchEmployees() {
    try {
      const res = await fetch('/api/department-head/employees');
      const data = await res.json();
      employees = data;
      renderEmployees(data);
    } catch (err) {
      console.error('Error fetching employees:', err);
    }
  }

  function renderEmployees(data) {
 const table = document.getElementById('employee-details-table');
 table.innerHTML = '';

 data.forEach(user => {
   const tr = document.createElement('tr');
   tr.innerHTML = `
     <td>${user.username}</td>
     <td>${user.department_name}</td>
     <td>${user.role}</td>
     <td>${user.email}</td>
     <td><button onclick="editUser(${user.id}, '${user.username}', '${user.role}',
'${user.email}')">Edit</button></td>
     <td><button onclick="deleteUser(${user.id})">Remove</button></td>
   `;
   table.appendChild(tr); // ✅ Add the row to the table body
 });
}
```

```
  function editUser(id, username, role, email) {
 const newUsername = prompt('Edit username:', username);
 if (!newUsername) return;

 const roleOptions = ['General Employee', 'Department Head'];
 const newRole = promptDropdown('Select role:', roleOptions, role);
 const newEmail = prompt('Edit email:', email);

 if (!newRole || !newEmail) return;

 fetch('/api/department-head/employees/update', {
   method: 'POST',
   headers: { 'Content-Type': 'application/json' },
   body: JSON.stringify({ id, username: newUsername, role: newRole, email: newEmail })
 }).then(() => fetchEmployees());
}

function promptDropdown(label, options, current) {
 const display = options.map((opt, i) => `${i + 1}. ${opt}`).join('\n');
 const choice = prompt(`${label}\n${display}\n(Current: ${current})`);
 const index = parseInt(choice);
 return isNaN(index) || index < 1 || index > options.length ? null : options[index -
1];
}


   // Initialize
   fetchBudgetStatus();
   fetchEmployees();
 </script>
</body>
</html>
```

department-head-expenditures.html

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <title>Department Expenditures</title>
</head>
<body style="background-color: #a2c2f4; font-family: Arial, sans-serif;">
```

```
 <h1 style="text-align: center; font-size: 32px; margin-top: 20px;">Department
Expenditures</h1>

 <h2 style="text-align: center; margin-top: 30px;">SPENDING TRACKER</h2>

 <table border="1" cellpadding="10" style="width: 80%; margin: 20px auto;
border-collapse: collapse; text-align: center;">
   <thead style="background-color: #4b7bec; color: white;">
     <tr>
       <th>Serial No.</th>
       <th>Expenditure Purpose</th>
       <th>Expenditure Amount</th>
       <th>Date</th>
       <th>Delete</th>
     </tr>
   </thead>
   <tbody id="expenditures-table"></tbody>
 </table>

 <div style="text-align: center; margin-top: 20px;">
   <button onclick="showAddForm()">Add Expenditure</button>
 </div>

 <!-- Hidden Add Expenditure Form -->
 <div id="add-form" style="display: none; margin: 20px auto; width: 50%; text-align:
center; background: #fff; padding: 20px; border-radius: 8px;">
   <h3>Add Expenditure</h3>
   <label>Expenditure Purpose: <input type="text"
id="expenditure-name"></label><br><br>
   <label>Expenditure Amount: <input type="number" id="expenditure-amount"
step="0.01"></label><br><br>
   <label>Date: <input type="date" id="expenditure-date"></label><br><br>
   <button onclick="addExpenditure()">Save</button>
   <button onclick="hideAddForm()">Cancel</button>
 </div>

 <script>
   let expenditures = [];

   // Fetch expenditures for this department
   async function fetchExpenditures() {
```

```
    try {
      const res = await fetch('/api/department-head/expenditures');
      const data = await res.json();
      expenditures = data;
      renderExpenditures(data);
    } catch (err) {
      console.error('Error fetching expenditures:', err);
    }
  }

  function renderExpenditures(data) {
    const table = document.getElementById('expenditures-table');
    table.innerHTML = '';
    data.forEach((exp, index) => {
      const tr = document.createElement('tr');
      tr.innerHTML = `
        <td>${index + 1}</td>
        <td>${exp.expenditure_name}</td>
        <td>${Number(exp.expenditure_amount).toLocaleString()}</td>
        <td>${new Date(exp.date).toLocaleDateString()}</td>
        <td><button
onclick="deleteExpenditure(${exp.expenditure_id})">Delete</button></td>
      `;
      table.appendChild(tr);
    });
  }

  function showAddForm() {
    document.getElementById('add-form').style.display = 'block';
  }

  function hideAddForm() {
    document.getElementById('add-form').style.display = 'none';
  }

  async function addExpenditure() {
    const name = document.getElementById('expenditure-name').value;
    const amount = document.getElementById('expenditure-amount').value;
    const date = document.getElementById('expenditure-date').value;

    if (!name || !amount || !date) {
      alert('Please fill all fields.');
```

```
      return;
    }

    const res = await fetch('/api/department-head/expenditures/add', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ expenditure_name: name, expenditure_amount: amount, date
})
    });

    if (res.ok) {
      hideAddForm();
      fetchExpenditures();
    } else {
      alert('Error adding expenditure.');
    }
  }

  async function deleteExpenditure(id) {
    if (!confirm('Are you sure you want to delete this expenditure?')) return;

    const res = await fetch('/api/department-head/expenditures/delete', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ id })
    });

    if (res.ok) {
      fetchExpenditures();
    } else {
      alert('Error deleting expenditure.');
    }
  }

  fetchExpenditures();
 </script>
</body>
</html>
```

General Employee Interface

employee-dashboard.html

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <title>Employee Dashboard</title>
 <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body style="background-color: #a2c2f4; font-family: Arial, sans-serif;">

 <h1 style="text-align: center; font-size: 32px; margin-top: 20px;">Employee
Dashboard</h1>

 <!-- Budget Approval Status Table -->
 <h2 style="text-align: center; margin-top: 30px;">BUDGET APPROVAL STATUS</h2>
 <table border="1" cellpadding="10" style="width: 80%; margin: 20px auto;
border-collapse: collapse; text-align: center;">
   <thead style="background-color: #4b7bec; color: white;">
     <tr>
       <th>Department</th>
       <th>Department Head</th>
       <th>Amount</th>
       <th>Approve/Deny</th>
       <th>Comments</th>
     </tr>
   </thead>
   <tbody id="budget-status-table" style="background-color: white;"></tbody>
 </table>

 <!-- Planned vs Actual Spending Graph -->
 <h3 style="text-align: center; margin-top: 40px;">Planned vs Actual Spending</h3>
 <canvas id="spendingChart" width="400" height="200" style="margin: 0 auto; display:
block; background-color: white; padding: 20px; border-radius: 8px;"></canvas>

 <script>
   async function fetchEmployeeDashboard() {
     try {
       const res = await fetch('/api/employee-dashboard-data');
       const data = await res.json();

       // === Fill Budget Approval Table ===
       const table = document.getElementById('budget-status-table');
       table.innerHTML = '';
```

```
      if (data.department) {
        const row = data.department;
        const tr = document.createElement('tr');
        tr.innerHTML = `
          <td>${row.department_name}</td>
          <td>${row.department_head}</td>
          <td>₹${Number(row.requested_budget).toLocaleString()}</td>
          <td>${row.budget_status || '—'}</td>
          <td>${row.admin_comments || '—'}</td>
        `;
        table.appendChild(tr);
      }

      // === Create Graph (Planned vs Actual) ===
      if (data.spending) {
        const planned = Number(data.spending.requested_budget);
        const actual = Number(data.spending.actual_spent);

        const ctx = document.getElementById('spendingChart').getContext('2d');
        new Chart(ctx, {
          type: 'bar',
          data: {
            labels: [data.spending.department_name],
            datasets: [
              {
                label: 'Planned Spending',
                data: [planned],
                backgroundColor: '#FF6384',
                borderColor: '#FF6384',
                borderWidth: 1
              },
              {
                label: 'Actual Spending',
                data: [actual],
                backgroundColor: '#36A2EB',
                borderColor: '#36A2EB',
                borderWidth: 1
              }
            ]
          },
          options: {
            scales: {
```

```
              y: { beginAtZero: true }
            }
          }
        });
      }
    } catch (err) {
      console.error('Error fetching employee dashboard data:', err);
    }
  }

  fetchEmployeeDashboard();
 </script>
</body>
</html>
```